

LLM의 기초 Transformer 이해하기

Hankyong National University
Applied Modeling and Simulation Lab (AMAS)
Presenter: Gyeong Ho Min
Adviser: Un Chang Jeoung

목차

1. Background
 - Recurrent Neural Network, RNN
 - Long short – term memory
 - Sequence to Sequence
 - Attention
2. Transformer
3. 초기 Chat GPT에서의 Transformer 활용
4. 결론 및 향후 계획

1. Background [Recurrent Neural Network, RNN]

- RNN은 Hidden Layer의 노드에서 Activation Function을 거쳐 나온 결과값을 Output Layer로 보내면서 다시 다음 Hidden Layer 노드 계산의 입력 값으로 보내는 신경망.
- 시계열 데이터와 같은 연속적인 정보를 처리할 때 사용되는 신경망

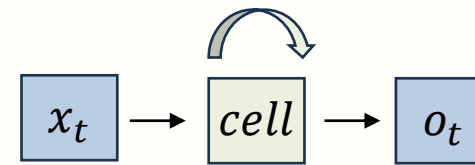


그림1. RNN 도식화

x = Input Layer의 입력 벡터
 o = Output Layer의 출력 벡터
 $cell$ = Hidden Layer에서 activation Function을 거쳐 결과를 내보내는 역할

First -Order-system

- 현재 시간의 상태가 이전 시간의 상태와, 현재 입력에 관여
$$h_t = f(h_{t-1}, x_t) \quad h = \text{상태}, x = \text{입력} \quad (1)$$

각 시간에서 hidden state를 통해 출력 o_t 생성
$$o_t = H(h_t) \quad H = \text{비선형함수라 가정}$$

특징: 과거든 현재든 같은 weight를 사용

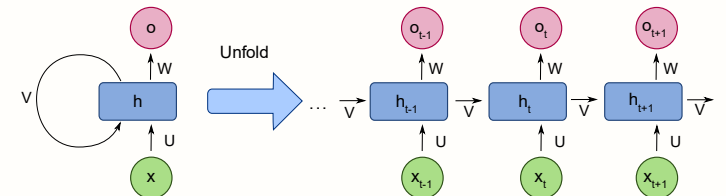
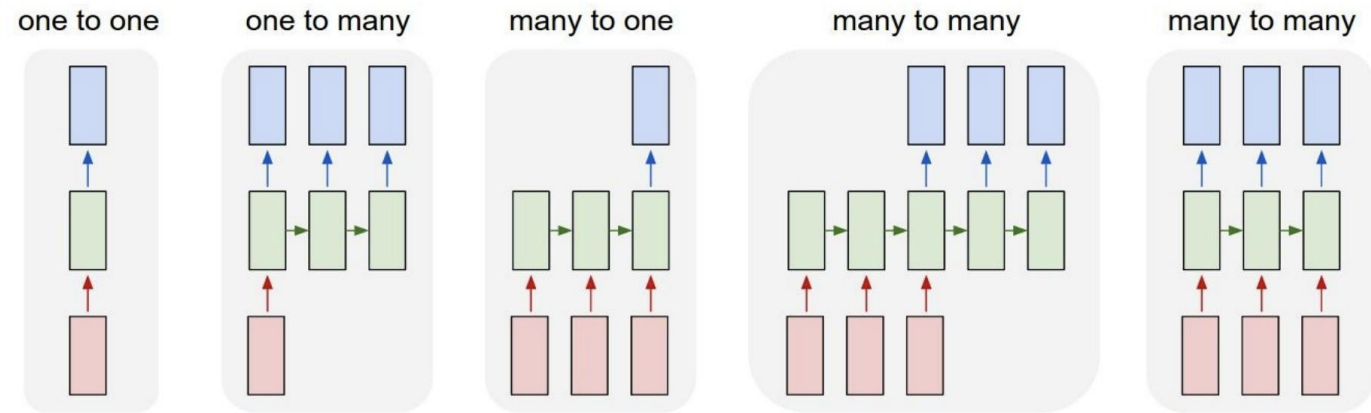


그림2. RNN 도식화 (2)

(1) Rumelhart, D. E., Hinton, G. E., & Williams, R. J., "Learning representations by back-propagating errors," Nature Publishing Group, vol. 323, no. 6088, pp. 533-536, 1986.

(2) https://commons.wikimedia.org/wiki/File:Recurrent_neural_network_unfold.svg

1. Background [Recurrent Neural Network, RNN]



RNN 유형 (3)

RNN의 문제점

Long-Term Dependency 학습의 어려움
Long sequency에서 성능 저하

계속 곱해지는 feedback loop로 곱해지는 값이 1 이하/이상 일 때

0 → Vanishing gradient
 ∞ → Exploding gradient



RNN의 문제점 해결방안

Gated RNNs: LSTM/GRU

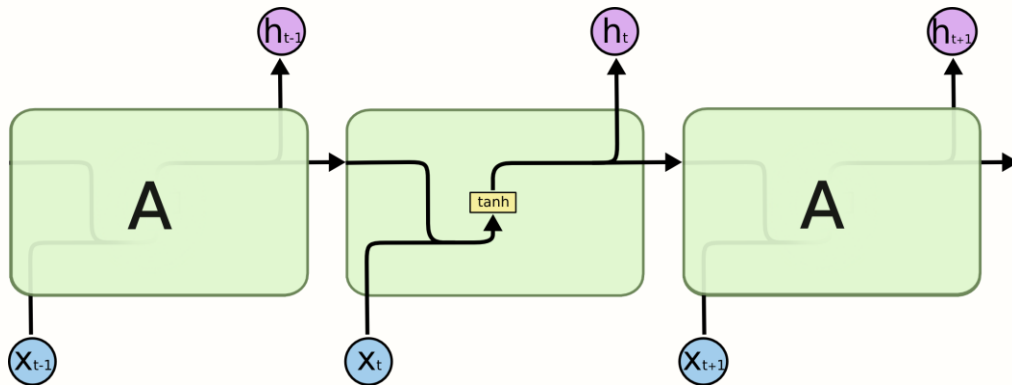
1. Background[Long short-term memory, LSTM]

LSTM 도입

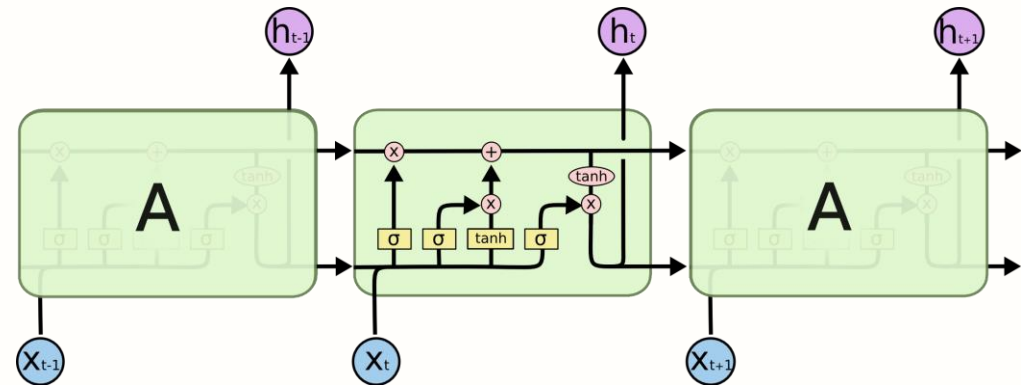
Cell state와 gate 구조를 통해 gradient를 안정적으로 전달

- Gradient flow를 제어할 수 있는 “밸브” 역할
- State space의 입력, 상태, 출력 구조는 동일
 - Gate 구조 추가
 - 4개의 MLP 구조

- Input gate
- Forget gate
- Cell state
- Output gate



RNN의 반복 모듈: 단 하나의 Layer (5)



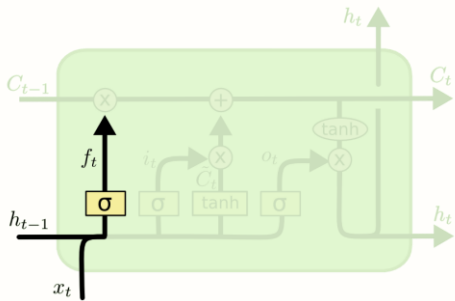
LSTM의 반복 모듈: 4개의 상호작용하는 Layer (5)

(4) S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," in *Neural Computation*, vol. 9, no. 8, pp. 1735-1780, 15 Nov. 1997, doi: 10.1162/neco.1997.9.8.1735.

(5) <https://dgkim5360.tistory.com/entry/understanding-long-short-term-memory-lstm-kr>

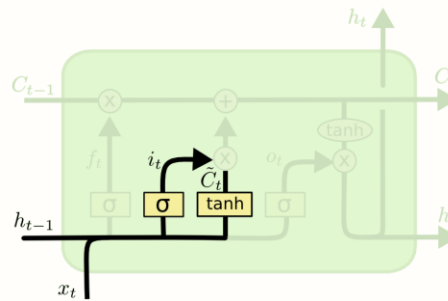
1. Background [Long short-term memory, LSTM]

1. Forget Gate – 이전 Cell state에서 불필요한 정보 제거
2. Input Gate – 새로운 정보 저장의 비율을 결정
3. Cell State 업데이트 – 이전 메모리와 신규 정보를 결합
4. Output Gate – 최종 출력(hidden state) 출력



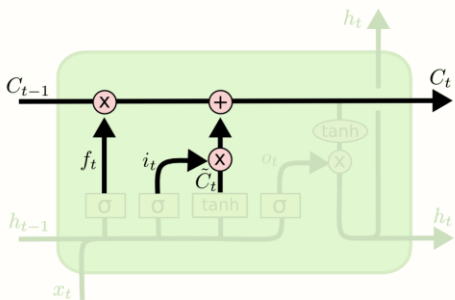
$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

LSTM의 Forget gate layer (5)



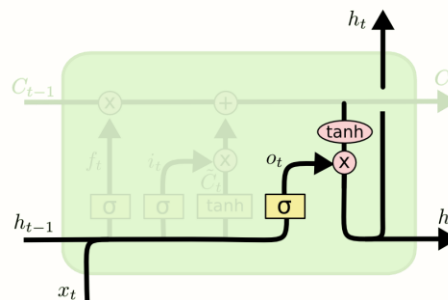
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

LSTM의 Input gate layer (5)



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

LSTM의 Cell state (5)



$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$
$$h_t = o_t * \tanh(C_t)$$

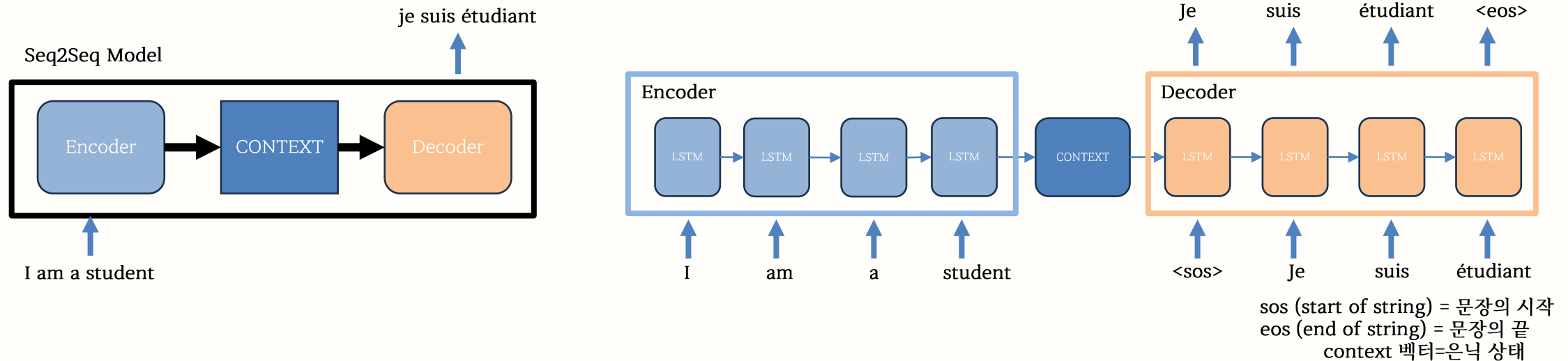
LSTM의 Output gate layer (5)

1. Background [Sequence to Sequence, Seq 2 Seq]

- Seq2Seq 모델은 주로 기계 번역에서 사용. (6) (7)
- 전통적인 RNN 기반 모델과 달리 가변 길이의 시퀀스를 처리할 수 있으며, 양방향의 문맥 정보를 효과적으로 활용할 수 있어 더 자연스러운 번역이 가능
- "I am a student" -> [Seq2Seq model] -> "je suis étudiant" (쥬 스위 에뛰디앙)

작동방식

1. 인코더는 "I am a student" 라는 입력문장을 받아 CONTEXT 벡터를 만들
2. CONTEXT 벡터는 "I am a student" 에 대한 정보를 압축
3. CONTEXT 벡터는 다시 디코더로 전달되며 디코더는 이를 활용하여 최종적으로 "je suis étudiant"라는 불어 문장을 생성



(6) I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to Sequence Learning with Neural Networks," *arXiv preprint arXiv:1409.3215*, 2014.

(7) <https://velog.io/@lighthouse97/seq2seq-%EB%AA%A8%EB%8D%B8%EC%97%90-%EB%8C%80%ED%95%9C-%EC%9D%B4%ED%95%B4>

1. Background [Attention]

Attention의 등장 배경⁽⁸⁾:

- Seq2Seq 모델은 입력 문장 전체 정보를 하나의 Context Vector에 압축하여 전달하기 때문에, 긴 문장의 경우 정보 손실이 발생하는 Bottleneck Problem이 존재
- 이러한 문제를 해결하기 위해 Decoder가 필요한 시점마다 Encoder의 hidden state를 직접 참고하는 Attention Mechanism이 제안
- 즉, 기존 Encoder-Decoder 기반 RNN 구조의 한계를 개선하기 위한 방법으로 Attention Mechanism이 등장 ⁽⁹⁾

해결방안

1. 출력 단어를 생성할 때 입력 문장 내에서 관련성이 높은 부분에 집중(attention)
2. 선택된 정보(Context Vector)와 이전에 생성된 단어(previous generated target word)를 기반으로 다음 목표 단어를 예측

⁽⁸⁾ D. Bahdanau, K. Cho, and Y. Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2015

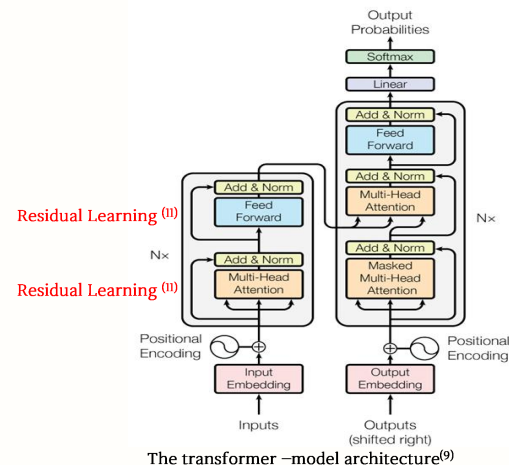
⁽⁹⁾ 세바스찬 라시카, 『밑바닥부터 만들면서 배우는 LLM』, 박해선 옮김, 길벗, 2025, p.83.

2. Transformer [Attention is all you need]

Transformer의 등장 배경⁽¹⁰⁾:

- RNN/LSTM 기반 Seq2Seq는 순차 연산 구조
- 긴 문장에서 Long-Term Dependency 문제 발생 병렬 연산이 어려워 학습 속도 제한
- Attention Mechanism으로 일부 문제 해결
- Transformer는 RNN 없이 Attention만 사용한 구조 제안

Transformer의 구조



Encoder

1. Input Embedding: 단어를 신경망이 이해할 수 있는 vector 형태로 변환.
2. Positional Encoding: 위치 정보를 가진 임베딩.
3. Multi-Head Attention: 하나의 Attention만 쓰는 것이 아니라 여러 개의 Attention을 병렬로 수행하는 구조
4. Residual Learning⁽¹¹⁾
 - Multi-Head Attention에서 잘못 학습하여도 원래 정보를 보존
 - Gradient 흐름 안정화
 - LayerNorm으로 학습 안정화
5. Feed Forward: 관계가 반영된 벡터를 더 좋은 특징 벡터로 변환

Positional Encoding

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$
$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

pos : 위치(position), i : 차원(dimension)

Decoder

1. Masked Multi-Head Self-Attention
 - 미래 단어 정보를 보지 못하도록 Mask 적용
 - 이전 단어들만 참고하여 다음 단어들만 참고하여 다음 단어 예측
 - 여러 개의 Attention을 병렬 수행하는 구조
2. Encoder-Decoder Attention
 - Encoder의 출력 정보를 참고
 - 입력 문장과 현재 생성 중인 단어의 관계 학습
 - 번역 및 문맥 연결 수행
3. Residual Learning
 - 각 서브 레이어 주변에 residual connection 적용
 - 원래 정보를 보존
 - Gradient 흐름 안정화
 - LayerNorm으로 학습 안정화
4. Feed Forward
 - Attention 결과를 비선형 변환
 - 특징 벡터를 더욱 풍부하게 표현
 - 단어 간 관계 정보를 강화
5. Linear + Softmax
 - 출력 벡터를 단어 확률로 변환
 - 가장 확률이 높은 다음 단어 예측

⁽¹⁰⁾ A. Vaswani et al., "Attention is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

⁽¹¹⁾ K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

2. Transformer [Attention is all you need]

FFN (Feed Forward Network)

$$FFN(x) = MAX(0, xW_1 + b_1)W_2 + b_2$$

- Attention 이후 특징 정보를 비선형 변환
- 두 개의 Linear Layer와 ReLU로 구성
- 각 위치(Position)에 독립적으로 적용 특징
- 표현 강화 및 학습 성능 향상
- 병렬 연산 가능

$xW_1 + b_1$: 첫 번째 선형 변환 수행

RELU : 비선형 활성화 적용

$W_2 + b_2$: 다시 선형 변환 수행

논문에서는 출력차원(512) → 2048(내부 층) → 512

2. Transformer [Attention is all you need]

Transformer Attention의 핵심 구조

Attention function

하나의 query와 key-value 쌍들의 집합을 출력(output)으로 매핑(mapping)하는 것으로 설명됨.

$Q(\text{Query}), K(\text{key}), V(\text{Value})$

$Q(\text{Query})$: 현재 기준이 되는 벡터

$K(\text{key})$: 비교 대상 벡터

$V(\text{Value})$: 실제 전달할 정보 벡터

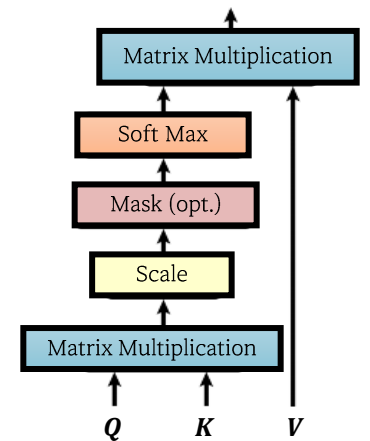
Scaled Dot-Product Attention

목적: Q 와 모든 K 사이의 내적을 계산하고, 각 값을 $\sqrt{d_k}$ 로 나눈 뒤, softmax함수를 적용하여 V 값들에 대한 가중치를 얻기 위해

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{K^T Q}{\sqrt{d_k}}\right) V$$

$$Q = \begin{pmatrix} | & | & | & | & | & | \\ Q_1 & Q_2 & Q_3 & Q_4 & \dots & Q_n \\ | & | & | & | & | & | \end{pmatrix}$$
$$K = \begin{pmatrix} | & | & | & | & | & | \\ K_1 & K_2 & K_3 & K_4 & \dots & K_n \\ | & | & | & | & | & | \end{pmatrix}$$

Dot-product의 분산이 d_k 에 비례하여 증가하므로, 표준편차인 $\sqrt{d_k}$ 로 scaling하여 값의 크기를 안정화



Scaled Dot-Product Attention⁽¹⁰⁾

(10) A. Vaswani et al., "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

2. Transformer [Attention is all you need]

Multi Head Attention

Multi-Head Attention 개념

- Query(Q), Key(K), Value(V)를 서로 다른 선형 변환(Linear Projection)으로 변환
- 여러 개의 Attention을 병렬 수행하는 구조
- 서로 다른 문맥 및 특징 관계를 동시에 학습 가능

핵심 수식

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$$

W_i^Q, K_i^K, W_i^V : 각 **head**마다 학습되는 가중치 행렬

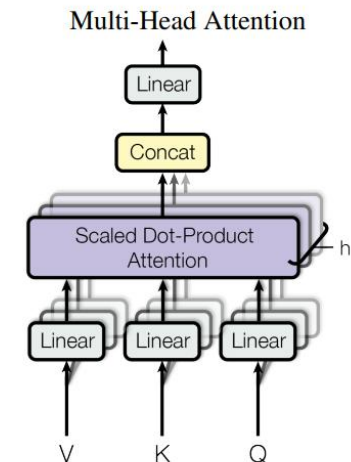
각 **head**가 서로 다른 특징 공간(subspace)을 학습

Multi-Head Attention 구조

$$MultiHead(Q, K, V) = Concat(head_1, head_2, \dots, head_h)W^O$$

(W^O = Output 가중치 행렬)

1. 각 **head**별 Attention 수행
2. 결과를 Concat으로 결합
3. 최종 Linear 변환 수행



Multi Head Attention⁽¹⁰⁾

(10) A. Vaswani et al., "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

2. Transformer [Attention is all you need]

Applications of Attention in our Model

Decoder Self-Attention

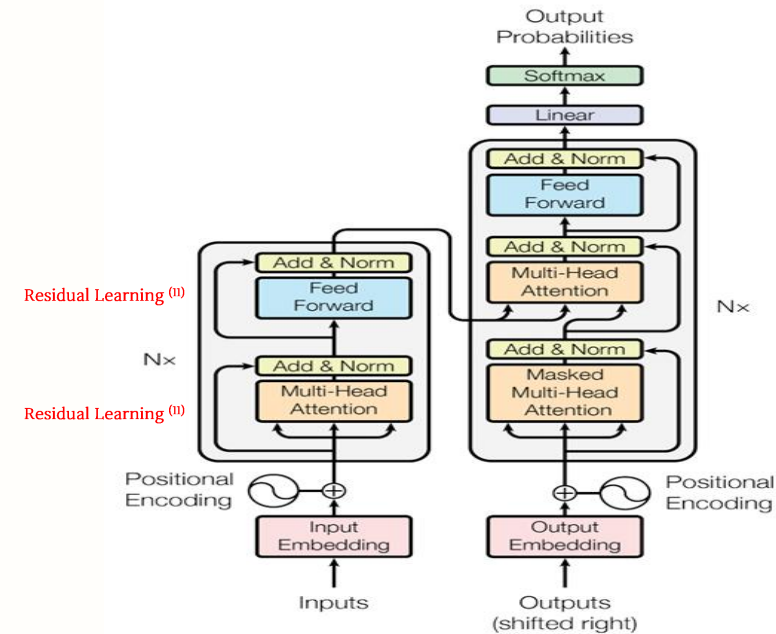
- 이전 단어들을 참고하여 다음 단어 예측
- 미래 단어 정보는 Masking 처리
- Auto-Regressive 방식 유지
- 문장 생성 과정의 순차성 보장

Encoder-Decoder Attention

- Decoder의 Query와 Encoder 출력의 Key, Value 사용
- 입력 문장과 출력 문장 간 관계 학습
- Decoder가 입력 문장의 중요한 단어를 참조하며 출력 생성
- Seq2Seq Attention 구조와 유사

Encoder Self-Attention

- 입력 문장 내부 단어들 간의 관계를 학습
- 각 단어가 모든 단어를 참조하여 문맥 정보 학습
- 장거리 의존성(Long-Term Dependency) 학습 가능
- 문장의 의미적 관계(Context) 파악



The transformer –model architecture⁽⁹⁾

(10) A. Vaswani et al., "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

3. GPT-1 : Generative Pre-Training 기반 Transformer 활용(12)

1. Pre-training

- 대규모 비지도 텍스트 데이터로 Language Modeling 수행
- 다음 단어 예측을 통해 문맥 및 언어 표현 학습
- Transformer Decoder 기반 구조 사용

2. Fine-tuning

- 사전학습된 모델을 downstream task에 미세조정
- QA, Text Classification, NLI 등 다양한 task 수행
- 구조 변경 없이 task-specific layer만 추가

GPT 논문의 핵심 :

- 하나의 Transformer 기반 언어모델을 다양한 NLP task에 전이 가능
- Pre-training된 언어 표현이 여러 task에서 높은 성능 달성
- 기존 LSTM 기반 접근보다 Long-range dependency 학습에 강점

Experimental Results

- 12개 task 중 9개 task에서 SOTA 달성
- Story Cloze : +8.9%
- RACE : +5.7%

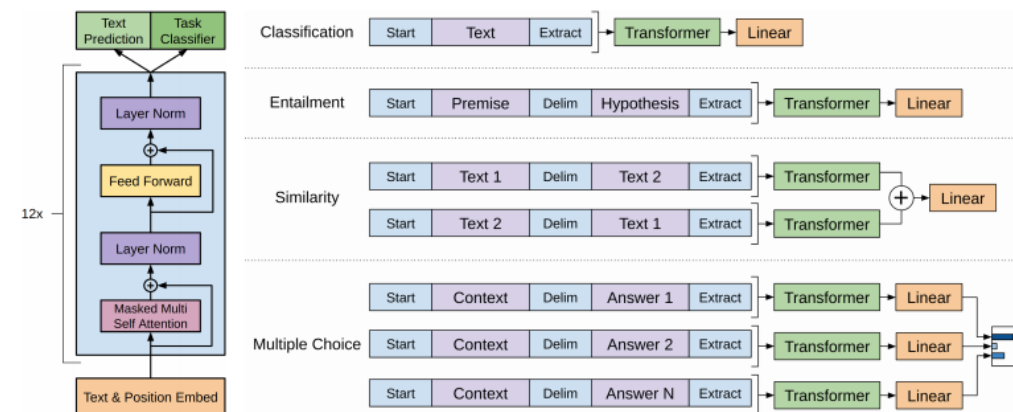


Figure 1: **(left)** Transformer architecture and training objectives used in this work. **(right)** Input transformations for fine-tuning on different tasks. We convert all structured inputs into token sequences to be processed by our pre-trained model, followed by a linear+softmax layer.

4. 결론 및 향후 계획

결론

- RNN → LSTM → Seq2Seq → Attention → Transformer로 발전
- Transformer는 병렬 연산과 Long-range dependency 문제를 개선
- GPT는 Transformer 기반 생성형 언어모델의 대표 사례

향후 계획

- GPT 및 최신 LLM 구조 추가 학습
- Engineering 분야에서의 Transformer 활용 가능성 탐구